

6주차 밤에 한 번 더 세는 이유 — 배치와 확정 통계

6주차 · 배치 처리

이 장의 수치는 모두 `practice/chapter6/` 의 코드를 실행해 얻은 값이다. 예시를 위해 지어낸 숫자는 없다. 인용한 값의 출처 파일은 출력 블록 첫 줄에 적었다.

한 가지 예외가 있다. Airflow 엔진이 직접 남긴 태스크 상태와 산출물 해시는 이 강의 환경에서 다시 돌릴 수 없어 **과거 실행이 남긴 산출물에서 옮긴 값**이며, 그 블록의 출처는

`practice/chapter6/data/output/` 으로 적었다.

- ☐ 5주차에서 스트리밍(streaming)은 지금 벌어지는 일을 빠르게 했지만, 워터마크(watermark)를 넘겨 도착한 이벤트를 폐기했음
 - 그 폐기는 소리를 내지 않았고, 상황판에 남은 수치는 근사치였음
- ☐ 이 장은 그다음 날 새벽에 같은 데이터를 다시 세는 일을 다룸
 - 배치(batch)는 느린 대신 마감까지 들어온 것을 전부 담고, 한 번 낸 수치를 바꾸지 않음
 - 그래서 기관이 제출하는 확정 통계는 배치가 만듦

5주차의 Spark와 이 장의 Airflow는 하는 일이 다르다

- **Spark**(5주차): 데이터를 실제로 계산하는 **분산 처리 엔진**이다. 한 대로 몇 시간 걸릴 집계를 여러 대에 나눠 끝낸다.
- **Airflow**(이 장의 선택 실습): 계산을 **언제·어떤 순서로 부를지 정하고, 그 결과를 상태로 적어 두는** 도구이며 계산 자체는 하지 않는다. 이런 도구를 워크플로 오케스트레이터(orchestrator)라 하고, 무엇을 맡는지는 5절에서 실행으로 확인한다.
- 그래서 둘은 대체 관계가 아니라 **층이 다르다**. 실무에서는 Airflow가 태스크 하나로 Spark 잡을 제출하고, 계산은 Spark가 하는 구조를 쓴다. 3주차 표 3.1이 둘을 같은 「처리」 칸에 적었지만, 역할은 계산(Spark)과 순서·시각·실패 기록(Airflow)으로 갈린다.
- 이 장이 Spark를 쓰지 않는 이유는 데이터가 사흘치 60건이어서다. 분산 계산이 필요한 규모가 아니므로 집계는 Python 표준 라이브러리가 맡는다. Spark를 쓰는 경우는 5주차 9절의 표 5.2에 정리했다.

1. 오늘의 질문

- ☐ 5주차에서 이미 실시간으로 세고 있는데, 밤에 왜 한 번 더 세는가?
- ☐ 원천 60건이 보고서에서 54건이 되었다면, 나머지 6건은 어디로 갔다고 답할 수 있는가?

- ☐ 배치가 중간에 실패했는데 보고서 파일이 만들어져 있다면, 무엇이 잘못된 것인가?
- ☐ 같은 날짜를 다시 계산했더니 수치가 달라졌다면, 어느 수치가 진짜인가?

2. 이 장에서 실행할 것

- ☐ 배치는 가게가 하루를 마감하는 절차와 항목이 하나씩 대응함

장부 마감	이 장의 배치
영업 중 계산대 화면에 뜨는 잠정 매출	스트리밍 상황판의 근사치(5주차)
문을 닫은 뒤 전표를 전부 다시 세어 만든 일계표	마감 후 전량을 재계산한 확정 통계(3절)
전표를 한 장도 버리지 않고 정상·기재누락·코드없음·중복으로 나눠 둠	정제의 네 갈래 출구(4절)
세어 본 합이 전표 뭉치와 안 맞으면 마감하지 않음	총계 보존식이 깨지면 배치를 실패시킴(5절)
일계표를 모아 월계표를 만들 때 전표를 다시 세지 않음	주간 롤업(rollup)은 일별 확정 산출물만 읽음(6절)

표 6.1 장부 마감과 배치의 대응

- ☐ 비유는 여기까지다
 - 장부는 사람이 마감하므로 합이 안 맞으면 조정 전표를 끼워 넣어 억지로 맞출 수 있음
 - 사전에 없는 지역 표기를 "가장 비슷한 자치구"로 바꿔 합을 맞추는 것은 데이터 조작이며, 이 장의 설계는 맞추는 대신 멈춤
 - 그 멈추는 장면이 5절에서 숫자로 나타남
- ☐ 이 장에서 실행할 코드는 넷이며 순서가 정해져 있음

순서	실행할 파일	하는 일	절	선행 조건
1	6-2-cutoff-scope.py	구간 경계 확인과 전수 중복 검사	3	없음
2	6-3-clean-accounting.py	표준 코드 매핑과 총계 보존식	4	없음
3	6-4-dag-order.py	태스크 순서·실패 전파·품질 게이트	5	없음
4	6-5-rerun-rollup.py	재실행 해시 비교와 주간 롤업	6	data/output/daily/ (저장소에 포함)
선택	6-1-daily-report-dag.py	같은 절차를 실제 Airflow 엔진으로	5-6	macOS·Linux·WSL2

시작 전 준비

- 1~4번은 표준 라이브러리만 쓰므로 설치할 패키지가 없음(실습 검증 환경은 Python 3.13)
- 선택 실습만 Airflow 3.3.0이 필요하고, Airflow는 Windows 네이티브를 지원하지 않음
 - 아래 한 줄이 운영체제·네트워크·패키지를 점검하고 가상환경을 준비함

```
python scripts/setup_practice.py 6
```

3. 배치의 마감은 달력이 정한다

- 같은 민원을 세는데 왜 두 시스템이 필요한지는 산출물의 성격에서 갈림
 - 상황판의 수치는 빨라야 하지만 나중에 바뀌어도 됨
 - 기관이 제출한 통계는 늦어도 되지만 제출한 뒤에 바뀌면 안 됨
 - 한 시스템으로 두 요구를 함께 만족시킬 수 없어 스트리밍과 배치를 나란히 둬
- 두 방식의 핵심 차이는 **마감을 정하는 주체**임
 - 스트리밍에서는 데이터가 마감을 정함
 - 5주차의 워터마크는 관측된 가장 늦은 이벤트 시각에서 허용 지연만큼을 뺀 시점을 마감으로 삼았음
 - 그 선을 넘겨 도착한 21:49 이벤트는 폐기되어 집계에 영영 들어가지 못했음
(practice/chapter5/data/output/ch5_late_event_report.json 의 dropped_by_watermark_cumulative)
 - 배치에서는 달력이 마감을 정함
 - "7월 1일분은 7월 2일 새벽에 마감한다"고 정해 두면, 그때까지 도착한 지연분은 늦었든 아니든 전부 그 구간에 들어감

□ 여기서 배치의 세 성질이 따라 나옴

- **확정성** — 구간 전체를 한 번에 계산하므로 산출물이 그 입력에 대한 최종 결과가 됨
 - 스트리밍이 실시간으로 낸 근사치를 사후에 검증하는 기준선 역할을 함
- **전체 조망** — 그날 데이터를 통째로 쥐고 있으므로 전수 중복 검사와 원천 대사가 성립함
 - 스트리밍은 흐르는 데이터의 창(window)만 보므로 창을 벗어난 쌍을 만나지 못함
- **재실행 가능성** — 실패하면 그 날짜를 통째로 다시 돌림
 - 다시 돌린 결과가 기존 산출물 위에 이중으로 쌓이면 안 되므로, 4주차의 멍등성(idempotency)이 배치의 전제로 되돌아옴
- 셋의 대가는 지연임. 일 배치라면 산출물이 최대 하루 늦고, 그 사이의 상황 판단은 여전히 스트리밍이 맞음

□ 첫 번째 코드를 실행함. 이 절의 숫자는 전부 이 명령의 출력임

```
cd practice/chapter6
python3 code/6-2-cutoff-scope.py
```

출처: practice/chapter6/results/6-2-cutoff-scope.log

[1] 입력 - 날짜 하나가 파일 하나다

2026-07-01.jsonl 물리 레코드 20건 첫 접수 08:00 마지막 접수 18:08
2026-07-02.jsonl 물리 레코드 18건 첫 접수 08:00 마지막 접수 17:04
2026-07-03.jsonl 물리 레코드 22건 첫 접수 08:00 마지막 접수 19:12
사흘 합계 60건

[2] 달력 마감 - 파일 안의 레코드가 모두 그 날짜인가

2026-07-01: 전부 같은 날짜
2026-07-02: 전부 같은 날짜
2026-07-03: 전부 같은 날짜
→ 구간의 경계를 데이터가 아니라 달력이 정함

[3] 전체 조망 - 하루치를 통째로 쥐면 전수 중복 검사가 성립한다

2026-07-02: C260702-001 가 두 번 - 08:00 과 11:12, 간격 192분
5주차 창 길이 10분 기준으로 같은 창에서 만나는가: 아니오

[4] 하루 안에서의 접수 분포 - 배치가 한 번에 보는 범위

2026-07-01: 접수 시각대 8시~18시, 시각대 11개에 걸침
2026-07-03: 접수 시각대 8시~19시, 시각대 12개에 걸침

[5] 지역 표기의 원문 - 정제 전에는 이렇게 들어와 있다

'강남구' 18건
'강남' 4건
None 2건
' 강남구 ' 1건
'강남 ' 1건

원문 표기 13가지 - 정제는 6-3에서 이어감

□ 화면에 나온 값에서 읽을 것

- **구간의 경계가 달력임** — 세 파일 어디에도 다른 날짜의 레코드가 섞여 있지 않음. "7월 1일분"이 무엇인지를 데이터의 도착 상황이 아니라 파일 이름이 정함
- **중복 두 건의 간격이 192분임** — 같은 민원 ID C260702-001 이 08:00과 11:12에 두 번 들어왔음. 5주차가 쓴 10분 창으로는 이 둘이 같은 창에서 만나지 못하므로 창 단위 처리로는 지울 수 없는 중복임
- **하루가 창 여럿에 흩어져 있음** — 7월 3일 접수는 8시부터 19시까지 시각대 12개에 걸쳐 있음. 배치는 그 전부를 한 번에 쥐므로 간격과 무관하게 전수 검사가 성립함
- **원문 표기가 13가지임** — 정제 전 지역 표기는 자치구 세 곳을 가리키는 데 13가지 형태로 들어와 있음. 4절이 여기서 출발함

- 두 수치가 다른 것 자체는 결함이 아니다
- 스트리밍의 근사치와 배치의 확정치는 마감 규칙이 다르므로 다를 수 있다.
- 결함인지 설계인지를 가르는 것은 왜 다른지가 문서로 설명되는가다. 설명되면 설계고, 설명되지 않으면 결함이다.

4. 표기가 흔들려도 총계는 맞아야 한다

- 정제에서 가장 위험한 코드는 처리하지 못한 레코드를 조용히 건너뛰는 코드임
 - 4주차의 유실도, 5주차의 워터마크 폐기도 스스로를 알리지 않았음
 - 정제 단계의 누락도 마찬가지여서, 보고서 총계가 원천보다 조금 작은데 아무도 모르는 상태가 됨
- 문제가 시작되는 자리는 지역명 표기임
 - 같은 강남구 민원이 "강남구", " 강남구 " (앞뒤 공백), "강남" (약칭), "강남 " (뒤 공백)으로 적혀 있음
 - 사람 눈에는 같은 곳이지만 이름을 키로 집계하면 한 자치구가 여러 줄로 쪼개짐
 - 그래서 이름 대신 행정표준코드(법정동코드)를 키로 씀
 - 코드를 쓰면 인구·예산 같은 다른 기관 데이터와도 같은 키로 붙일 수 있음
- 정제는 모든 입력 레코드를 네 갈래 중 하나로 보내고 어디로 갔는지 기록함
 - 매핑 성공(mapped) — 표준 코드가 붙어 집계에 들어감
 - 지역 미기재(missing) — 지역 필드가 비어 있음. 건수와 민원 ID를 따로 기록함
 - 매핑 실패(unmapped) — 사전에 없는 표기임. 원문 그대로 따로 기록함
 - 중복 제거(dup_removed) — 같은 민원 ID의 재전송은 먼저 온 것만 남김(4주차의 원칙)
- 네 갈래의 합이 입력과 맞는지 검사하는 식이 **총계 보존식**임

```
physical = mapped + missing + unmapped + dup_removed
valid     = physical - dup_removed = mapped + missing + unmapped
```

- 두 번째 코드를 실행함. 6-1의 Airflow DAG와 같은 정제 규칙을 Airflow 없이 적용함

```
python3 code/6-3-clean-accounting.py
```

출처: practice/chapter6/results/6-3-clean-accounting.log

[2] 지역명을 그대로 키로 쓰면 - 사흘치 원문 표기 기준

집계 줄 수 13줄

강남 계열만 4줄로 쪼개짐: ' 강남구 ' 1건, '강남' 4건, '강남 ' 1건, '강남구' 18건

강남 계열 합 24건 - 어느 줄도 단독으로는 이 수를 보이지 못함

[3] 정규화 후 표준 코드를 키로 쓰면

집계 줄 수 3줄

11440 마포구: 16건

11620 관악구: 15건

11680 강남구: 23건

강남구 23건 - [2]의 강남 계열 합보다 1건 적음(중복 점수 제거분)

[4] 일별 정제 회계 - $\text{physical} = \text{mapped} + \text{missing} + \text{unmapped} + \text{dup_removed}$

날짜	physical	dup	valid	mapped	missing	unmapped	보존
----	----------	-----	-------	--------	---------	----------	----

2026-07-01	20	0	20	18	1	1	OK
------------	----	---	----	----	---	---	----

검산 $20 = 18 + 1 + 1 + 0$ 공백 보정 1건, 약칭 보정 4건

2026-07-02	18	1	17	16	0	1	OK
------------	----	---	----	----	---	---	----

검산 $18 = 16 + 0 + 1 + 1$ 공백 보정 1건, 약칭 보정 3건

2026-07-03	22	0	22	20	1	1	OK
------------	----	---	----	----	---	---	----

검산 $22 = 20 + 1 + 1 + 0$ 공백 보정 1건, 약칭 보정 3건

[5] 사흘 합계 - $60 = 54 + 2 + 3 + 1$ 보존 OK

미기재 C260701-019, C260703-021

미매핑 '판교', '성남시', '김포'

중복제거 C260702-001

[6] 조용히 건너뛰는 코드와 대조 - 매핑 실패를 기록하지 않으면

보고서에 적히는 총계 54건

원천 물리 레코드 60건

설명되지 않는 차이 6건

예외 없음, 종료 코드 0 - 차이가 로그에도 산출물에도 남지 않음

□ 화면에 나온 값에서 읽을 것

○ **키를 바꾸면 13줄이 3줄이 됨** — 이름을 키로 쓰면 강남 계열만 4줄로 쪼개지고, 어느 줄도 단독으로는 강남구 전체를 보이지 못함. 표준 코드를 키로 쓰면 세 줄로 모임

○ **강남 계열 합 24건과 매핑된 강남구 23건이 1건 다름** — 사라진 1건은 7월 2일의 재전송 C260702-001 임. 두 수가 다른 이유가 산출물 안에 적혀 있으므로 확인됨

- **사흘 전체가 60 = 54 + 2 + 3 + 1로 남김없이 설명됨** — 원천 60건 중 어느 것도 행방이 빈칸으로 남지 않음
- **조용히 버리면 차이 6건이 어디에도 남지 않음** — 매핑 실패를 세지 않는 코드는 보고서에 54건만 적고 종료 코드 0으로 끝남. 감사에서 가장 곤란한 결함이 이 형태임
- **미기재와 미매핑을 합쳐 "실패 2건"으로 보고하지 않음**
 - 미기재는 접수 화면이 지역을 받지 못했다는 뜻이므로 조치 대상이 접수 시스템임
 - 미매핑은 값은 들어왔는데 우리 사전에 없다는 뜻이므로 조치 대상이 사전임
 - 실제로 걸린 판교·성남시·김포는 오타가 아니라 서울 자치구 사전 밖의 유효한 지명일 수 있음
 - 관할이 넓어졌는지 접수 오류인지는 사람이 판단하며, 두 항을 합치면 그 판단에 필요한 정보가 사라짐

5. DAG와 품질 게이트 — 틀린 보고서를 아예 만들지 않는다

- 정제·집계·보고서 쓰기를 한 파일에 차례로 이어 붙이면, 매일 돌리기 시작한 뒤에 곧 막힘
 - 어제 새벽 배치가 집계 도중에 죽었을 때 어디서부터 다시 돌려야 하는지 알 수 없음
 - 정제가 실패했는데 보고서 쓰는 부분이 어제 파일을 읽고 그냥 돌아 버렸는지도 알 수 없음
 - 둘 다 같은 데서 나옴 — **어느 단계까지 끝났는지가 어디에도 적히지 않음**
- 이 문제를 푸는 도구를 워크플로 오케스트레이터(workflow orchestrator)라 함
 - 하는 일은 하나임 — 절차를 **단계(태스크, task)**로 쪼개고, 단계 사이의 순서를 코드에 적게 함
 - 쪼갬 단계를 점으로, 순서를 화살표로 그리면 그림이 하나 나옴. 그 그림을 **DAG**라 함
 - Directed Acyclic Graph, 우리말로 방향 비순환 그래프임
 - **방향**은 화살표가 한쪽으로만 간다는 뜻임. "정제가 끝나야 집계한다"가 그 한쪽임
 - **비순환**은 화살표를 따라가도 제자리로 돌아오지 않는다는 뜻임. 돌아오면 두 단계가 서로를 기다리며 영영 멈춤
 - 순서를 적어 두면 나머지는 엔진이 함 — 적힌 순서대로 부르고, 앞 단계가 실패하면 뒤를 안 부르고, 정해진 시각에 깨우고, 실패한 단계를 다시 돌림

```
# practice/chapter6/code/6-1-daily-report-dag.py 의 순서 선언
t_validate >> t_clean >> [t_aggregate, t_quality] >> t_report
```

- 단계를 어디서 끊을지는 두 가지가 정함
 - 하나는 **다시 돌리는 단위**임
 - 실패하면 그 단계부터 다시 돌리므로, 정제와 집계를 한 단계로 묶으면 집계에서 죽을 때마다 정제를 처음부터 다시 함
 - 다른 하나는 **실패한 이유를 가릴 수 있는가**임
 - 입력 검증은 파일이 없어서 실패하고, 품질 점검은 데이터가 규칙을 어겨서 실패함

- 둘을 한 단계로 묶으면 「실패」라는 상태만 남아 어느 쪽인지 가릴 수 없음. `check_quality` 를 집계에서 떼어 낸 이유가 이것임

□ `check_quality` 가 하는 일은 4절의 총계 보존식 검사임

○ 등식이 깨지면 이 단계가 스스로 오류를 내고 멈춤. 그러면 뒤에 오는 `write_report` 는 아예 시작하지 않음

○ **틀린 보고서를 만든 뒤 회수하는 것이 아니라 애초에 만들지 않는 것** — 이것이 품질 게이트가 노리는 것임

□ 세 번째 코드를 실행함. 같은 그래프를 표준 라이브러리만으로 만든 작은 실행기로 돌림

```
python3 code/6-4-dag-order.py
```

출처: practice/chapter6/results/6-4-dag-order.log

[1] 의존성 선언에서 나오는 실행 순서

validate_input → clean_and_map → aggregate_daily → check_quality → write_report

순서가 정해진 쌍 9개 / 정해지지 않은 쌍 1개

순서 없음: aggregate_daily 와 check_quality - 실행마다 선후가 달라질 수 있음

[2] 2026-07-01 - 입력이 있는 날

dagrun=success

validate_input success

clean_and_map success

aggregate_daily success

check_quality success

write_report success

남은 산출물 6개: validated.json, cleaned.jsonl, quality.json, daily_summary.json, quality_check.json, report.md

[3] 2026-07-04 - 입력 파일이 없는 날

dagrun=failed

validate_input failed FileNotFoundError: 입력 파일 없음: 2026-07-04.jsonl

clean_and_map upstream_failed validate_input 이(가) 끝나지 않음

aggregate_daily upstream_failed clean_and_map 이(가) 끝나지 않음

check_quality upstream_failed clean_and_map 이(가) 끝나지 않음

write_report upstream_failed aggregate_daily 이(가) 끝나지 않음

남은 산출물 0개: 없음

[4] 정제에 누락 결함을 심은 2026-07-01 - 품질 게이트가 막는가

주입한 결함: 매핑 실패 건수를 세지 않고 버린다(drop_silently=True)

dagrun=failed

validate_input success

clean_and_map success

aggregate_daily success

check_quality failed ValueError: 총계 보존 실패: $20 \neq 18 + 1 + 0 + 0$

write_report upstream_failed check_quality 이(가) 끝나지 않음

남은 산출물 4개: validated.json, cleaned.jsonl, quality.json, daily_summary.json

□ 화면에 나온 값에서 읽을 것

- 순서를 적어 주지 않은 쌍이 하나 있음 — aggregate_daily 와 check_quality 는 둘 다 정제 결과만 있으면 되고 서로를 기다릴 이유가 없어 순서를 적지 않았음. 그래서 어느 쪽이 먼저 돌지는 실행마다 달라질 수 있고, 이것은 오류가 아님. 적지 않은 순서는 엔진이 지켜 주지 않음

- **입력이 없는 날 실제로 돌아간 단계는 하나뿐임** — 나머지 넷은 돌다가 실패한 것이 아니라 `upstream_failed` (앞 단계가 안 끝남)로 **아예 시작하지 않았음**. 예측을 "다섯 개 모두 실패"로 적었다면 여기서 갈림
 - **산출물이 0개임** — 뒤 단계가 돌지 않았으므로 절반쯤 쓰다 만 보고서가 남지 않음. 그런 파일이 남으면 다음 주 집계에 섞여 들어감
 - **누락 결함을 심으면 게이트에서 멈춤** — 매핑 실패 1건을 세지 않게 바꾸자 $20 \neq 18 + 1 + 0 + 0$ 으로 등식이 깨지고 `check_quality` 가 멈췄음. 집계 파일까지는 만들어졌지만 `report.md` 는 없음
 - **실패가 기록으로 남음** — 4주차의 유실과 5주차의 폐기는 흔적을 남기지 않았지만, 이 실패는 단계마다 상태가 적힘. 그 기록이 없으면 "돌지 않았다"와 "돌 필요가 없었다"를 가릴 수 없음
- 같은 그래프를 실제 Airflow 3.3.0 엔진으로 돌린 적이 있고, 그 기록에도 같은 상태가 남아 있음

출처: `practice/chapter6/data/output/ch6_batch_run_report.json` (과거 실행 산출물, start 필드 생략)

```
"failure_run": {
  "dagrun_state": "failed",
  "logical_date": "2026-07-04",
  "tasks": [
    {"task_id": "validate_input", "state": "failed"},
    {"task_id": "clean_and_map", "state": "upstream_failed"},
    {"task_id": "aggregate_daily", "state": "upstream_failed"},
    {"task_id": "check_quality", "state": "upstream_failed"},
    {"task_id": "write_report", "state": "upstream_failed"}
  ]
}
```

- **입력 파일 없음은 재시도로 개선되지 않는 실패다**
- 원천이 몇 분 늦는 상황이라면 재시도가 파일 도착을 기다리는 장치가 된다.
- 파일이 아예 오지 않는 날이라면 재시도는 실패 통보를 늦추기만 한다. 어느 쪽인지는 원천의 지연 패턴이 정한다.

□ 마지막으로 **logical date**는 그 실행이 처리하는 데이터 구간의 날짜임

- 7월 1일 데이터를 처리한 실행의 logical date는 2026-07-01이지만 실제로 돌아간 날은 다를 수 있음
- 2주차의 이벤트 시간과 처리 시간 구분이 배치로 오면 "데이터의 날짜"와 "실행된 시각"의 구분이 됨
- 이 이름표가 있어야 "7월 1일 실행을 다시 돌려라"가 어느 입력을 읽고 어느 산출물을 덮어쓸지 확정된 명령이 됨

6. 재현성과 롤업 — 다시 돌리면 글자 하나까지 같은 파일이 나와야 한다

- 배치를 다시 돌릴 수 있다는 말은, 다시 돌린 결과가 처음 것과 같을 때만 뜻이 있음
 - 감사에서 "그 통계를 다시 만들어 보라"는 요구를 받았는데 값이 달라지면 어느 수가 맞는지를 두고 다투게 됨
 - 그래서 산출물의 **해시**를 적어 두고, 같은 날짜를 다시 돌린 뒤 두 해시를 맞대어 봄
 - 해시는 파일 내용을 짧은 글자열로 줄인 값임. 내용이 한 글자만 달라도 값이 크게 달라지므로, 두 파일이 같은지를 한 줄로 가릴 수 있음(3주치의 SHA-256)
 - 아래 로그에 나오는 「같은 바이트인가」가 이 비교임. **바이트**는 파일에 담긴 글자를 하나씩 세는 단위이므로, 「바이트가 같다」는 **글자 하나까지 같다**는 뜻임
- 글자 하나까지 같은 파일은 저절로 나오지 않음. 코드를 그렇게 짜야 나오고, 세 가지를 지킴
 - **보고서에 「지금 몇 시」를 적지 않음**
 - 생성 시각을 한 줄 넣으면 그 줄이 돌릴 때마다 달라지고, 한 글자만 달라도 파일 전체의 해시가 달라짐
 - 그러면 집계한 수가 똑같아도 「다시 만들어 같은 값이 나왔다」를 해시로 보일 수 없음. 아래 [2]가 그 장면임
 - 시각을 남겨야 하면 보고서 본문이 아니라 실행 기록에 적음. 보고서에는 **입력에서 계산한 것만** 넣음
 - **집계 결과를 적는 순서를 내용이 정하게 함**
 - 지역별 건수를 파일에 쓸 때 어느 지역을 먼저 쓸지를 「입력에 나온 순서」에 맡기면, 담긴 수는 같은데 줄 순서가 달라 해시가 갈림
 - 그래서 적기 전에 지역 코드 순으로 줄을 세우고(`sorted`), 한 줄 안의 항목 이름도 같은 순서로 고정함(`sort_keys=True`)
 - 둘 다 `6-4-dag-order.py`의 집계 태스크에 들어 있음
 - **산출물을 이어 붙이지 않고 덮어쓰**
 - 7월 1일을 두 번 돌렸을 때 이어 붙이면 같은 수가 두 번 들어가 건수가 두 배로 보임
 - 덮어쓰면 몇 번 돌려도 그 날짜의 파일은 한 벌이고 내용이 같음
 - 4주치의 UPSERT가 같은 키의 행을 새로 쌓지 않고 고쳐 쓴 것과 같은 설계임. 파일에서는 덮어쓰기가 그 일을 함
- **롤업(rollup)**은 같은 자료를 하루 단위에서 한 주 단위로 올려 다시 합하는 것임
 - 지킬 규칙은 하나임 — **한 주 집계는 하루 집계의 확정된 결과만 읽고, 원천을 다시 읽지 않음**
 - 주간 쪽이 원천을 다시 정제하면 같은 계산을 두 번 하게 됨. 게다가 정제 규칙이 한쪽만 달라지는 순간 하루치 합과 주간 값이 어긋남
 - 어느 날짜가 확정되었는지는 `_SUCCESS`라는 **빈 파일 하나**가 가림
 - 하루치 배치의 첫 단계가 시작할 때 이 파일을 지우고, 마지막 단계까지 성공했을 때만 다시 만들
 - 그래서 이 파일이 있다는 것은 **그날 배치가 품질 게이트까지 통과해 끝났다는** 뜻임
 - 주간 쪽은 이 파일이 있는 날짜만 합산하고, 없는 날짜는 결측으로 적음

□ 네 번째 코드를 실행함

```
python3 code/6-5-rerun-rollup.py
```

```
# 출처: practice/chapter6/results/6-5-rerun-rollup.log
```

[1] 2026-07-01 보고서를 두 번 만들어 파일 해시를 비교 - 입력만의 함수일 때

1회차 3e2de6b0dd9b5813...

2회차 3e2de6b0dd9b5813...

같은 바이트인가: 예

[2] 같은 보고서에 생성 시각 한 줄을 넣으면

달라진 줄: '- 생성 시각: ...' 한 줄 (실행할 때마다 값이 바뀜)

같은 바이트인가: 아니오

→ 집계 결과가 같아도 해시 비교로는 '재현됐다'를 보일 수 없게 됨

[3] 주간 창 2026-06-29 ~ 2026-07-05 - _SUCCESS 마커로 확정 여부를 판별

2026-06-29 마커 없음 일별 요약 파일 없음

2026-07-01 마커 있음 일별 요약 파일 있음

2026-07-02 마커 있음 일별 요약 파일 있음

2026-07-03 마커 있음 일별 요약 파일 있음

2026-07-04 마커 없음 일별 요약 파일 없음

합산 대상 3일, 결측 4일: 2026-06-29, 2026-06-30, 2026-07-04, 2026-07-05

[4] 일별 확정 산출물만 합산한 주간 값 - 원천을 다시 읽지 않음

총계 보존 60 = 54 + 2 + 3 + 1 OK

지역별 마포구 16, 관악구 15, 강남구 23

유형별 도로 21, 소음 15, 청소 8, 환경 10

검산 지역별 합 54 / 유형별 합 54 / 매핑 54

[5] 주간 쪽이 원천을 다시 정제하는데 정제 규칙만 구버전이면

구버전 사전: 약칭 ['강남', '마포'] - '관악' 항목이 아직 없음

원천 재정제 결과 매핑 52건 / 미매핑 5건

일별 합산 결과 매핑 54건 / 미매핑 3건

같은 주에 대해 두 보고서가 매핑 2건 차이

두 계산 모두 총계 보존은 성립함 - 각자 안에서는 합이 맞아 오류로 잡히지 않음

□ 화면에 나온 값에서 읽을 것

- 같은 입력으로 두 번 만든 보고서의 해시가 같음 — 3e2de6b0... 이 두 번 나왔음. "다시 해 봤더니 비슷했다"가 아니라 글자 하나까지 같음을 한 줄로 보일 수 있음

- **생성 시각 한 줄이 그것을 깨뜨림** — 집계한 수가 완전히 같은데도 해시가 달라짐. 계산이 맞는 것과, 해시로 견줄 수 있게 쓰인 것은 다른 문제임
- **_SUCCESS 파일이 있는 날만 합산됨** — 주간 창 7일 중 3일만 합산되고 나머지 4일은 결측으로 적혔음. 결측을 적지 않고 3일치만 합하면 "이번 주 민원 60건"이 7일치처럼 읽힘
- **지역별 합과 유형별 합이 모두 54로 매핑 건수와 맞음** — 주간 단계에서도 검산이 성립함. 이 검산이 없으면 합치는 과정에서 새로 생긴 누락을 발견할 수 없음
- **같은 주를 두 방법으로 세면 매핑 건수가 2건 어긋남** — 일별 확정 산출물을 합한 값과, 주간 쪽이 원천을 다시 정제한 값이 같림
 - 두 방법의 차이는 사전 한 항목뿐임. 주간 쪽 사전에는 강남·마포만 있고 관악이 없어 관악으로 적힌 민원을 표준 코드로 바꾸지 못함
 - 일별 합산은 매핑 54건·미매핑 3건, 원천 재정제는 매핑 52건·미매핑 5건임. **2건이 매핑에서 미매핑으로 옮겨 갔었을 뿐 어디로도 사라지지 않음**
 - 그래서 두 계산 모두 자기 안에서는 총계 보존식이 성립함. 4절의 검산은 **한 계산 안에서** 입력과 출구의 합을 맞추는 장치이므로, 다른 계산과 2건 어긋난 것은 보지 못함
 - 보고서를 받아 보는 쪽에서는 두 수가 모두 검산을 통과한 값이어서 어느 쪽이 맞는지 가릴 근거가 없음
 - 이 어긋남은 검사를 하나 더 붙여 막는 것이 아니라 구조로 막음 — 주간이 원천을 다시 읽지 않고 일별 확정 산출물만 합하면, 규칙이 갈릴 자리 자체가 생기지 않음
- 7월 1일을 실제 Airflow 엔진으로 다시 돌린 적이 있고, 그 기록에도 같은 결과가 남아 있음
 - `daily_summary.json` 과 `report.md` 의 해시가 다시 돌리기 전후로 같았고, `identical` 이 `true` 로 적혔음 (`practice/chapter6/data/output/ch6_batch_run_report.json` 의 `idempotency_check`)

- 재현성은 한 번 확인하고 끝나는 성질이 아니다
- 정제 규칙이나 보고서 서식을 고치면, 그날부터 같은 입력에서 다른 파일이 나온다.
- 일부러 고친 것이면 "확정치의 계산 규칙이 바뀐 날짜"로 적어 두고, 모르는 사이에 달라진 것이면 사고다. 해시 비교가 그 둘을 가른다.

7. 핵심 요약

- 1. 「7월 1일분」이 무엇인지는 데이터가 아니라 달력이 정한다. (3절)
 - 날짜 하나가 파일 하나이고, 세 파일 모두 다른 날짜의 접수가 섞이지 않았음
 - 마감 시각까지 들어온 것은 늦게 왔어도 그 날짜에 들어감. 5주차처럼 늦었다고 버리지 않는 것이 배치가 확정 통계를 만드는 이유임
- 2. 배치는 하루치를 한 번에 보므로, 창 하나만 보는 처리가 못 하는 일을 한다. (3절)
 - 7월 2일에 같은 접수번호가 08:00과 11:12에 두 번 들어왔고 간격이 192분임. 5주차의 10분 창에서는 두 건이 같은 창에 들어오지 못함

- 하루를 통째로 놓고 보면 그 둘이 한 묶음 안에 있으므로 중복을 지울 수 있음. 느린 대신 넓게 보는 것이 배치가 얻는 것임

□ 3. 정제된 민원은 한 건도 행방을 비워 두지 않고, 합이 맞는지 식으로 검사한다. (4절)

- 사흘치가 $60 = 54 + 2 + 3 + 1$ 로 전부 설명됐고, 날짜별로도 20·18·22가 각각 등식을 만족했음
- 매핑 실패를 세지 않는 코드는 보고서에 54건만 적고 종료 코드 0으로 끝남. 원천 60건과의 차이 6건이 로그에도 산출물에도 안 남는 것이 감사에서 가장 곤란한 결함임

□ 4. 집계 키를 지역 이름에서 표준 코드로 바꾼다. (4절)

- 이름을 키로 쓰면 집계 키가 13줄이 되고 강남 계열만 '강남구' 18건·'강남' 4건·'강남구' 1건·'강남' 1건 네 줄로 쪼개져, 어느 줄도 강남구 전체를 보이지 못함
- 표준 코드를 키로 쓰면 3줄로 모임. 표기가 흔들리는 원천에서는 사람이 읽는 이름을 키로 쓰지 않음

□ 5. 미기재와 미매핑을 합쳐 「실패 2건」으로 적지 않는다. (4절)

- 미기재는 접수 화면이 지역을 받지 못한 것이라 고칠 곳이 접수 시스템이고, 미매핑은 값은 들어왔는데 사전에 없는 것이라 고칠 곳이 사전임
- 실제로 걸린 판교·성남시·김포는 오타가 아니라 서울 자치구 사전 밖의 지명일 수 있음. 비슷한 자치구로 끼워 맞춰 합을 채우는 것은 데이터 조작임

□ 6. 태스크를 어디서 쪼갬지는 다시 돌리는 단위와 실패原因的의 구분이 정한다. (5절)

- 정제와 집계를 한 태스크로 묶으면 집계에서 죽을 때마다 정제를 처음부터 다시 함. 입력 검증과 품질 점검을 묶으면 「파일이 없어서」와 「규칙을 어겨서」를 상태만 보고 가릴 수 없음
- 선언하지 않은 순서는 엔진이 지켜 주지 않음 — `aggregate_daily` 와 `check_quality` 는 서로를 기다리지 않으므로 선후가 실행마다 달라질 수 있고, 그것은 오류가 아님

□ 7. 품질 게이트는 틀린 보고서를 회수하지 않고 애초에 만들지 않는다. (5절)

- 입력 파일이 없는 날 실제로 실행된 태스크는 `validate_input` 하나였고 나머지 넷은 `upstream_failed` 로 남아 산출물이 0개였음
- 누락 결함을 십자 $20 \neq 18 + 1 + 0 + 0$ 에서 멈춰 집계 파일까지만 남고 `report.md` 는 만들어지지 않았음. 절반쯤 쓰다 만 보고서를 남기지 않는 것이 게이트의 목적임

□ 8. 같은 입력을 다시 돌리면 글자 하나까지 같은 파일이 나와야 한다. (6절)

- 생성 시각을 넣지 않은 보고서는 두 번 만들어도 해시가 `3e2de6b0...` 으로 같았고, 시각 한 줄을 넣자 집계 결과가 똑같은데도 해시가 달라졌음
- 해시가 같아야 「다시 만들어 같은 값이 나왔다」를 한 줄로 보일 수 있음. 계산이 맞는 것과, 비교할 수 있는 형태로 적힌 것은 다른 문제임

□ 9. 주간 집계는 원천을 다시 읽지 않고 일별 확정 산출물만 합한다. (6절)

- `_SUCCESS` 마커가 있는 3일만 합산하고 나머지 4일은 결측으로 적었음. 결측을 안 적으면 3일치 합이 7일치처럼 읽힘
- 주간 쪽만 원천을 다시 정제하자 사전 한 항목 차이로 매핑이 54건과 52건으로 갈렸음. 두 수가 모두 자기 검산을 통과하므로 어느 쪽이 맞는지 가릴 근거가 없음

8. 과제

☐ 실습을 순서대로 실행해 만든 산출물 네 개를 LMS에 제출한다

1. 표 6.2 예측표 — 실행 결과와 대조해 채운 최종본. 텍스트든 이미지든 형식은 무방하다.

2. `practice/chapter6/results/6-3-clean-accounting.log`

3. `practice/chapter6/results/6-4-dag-order.log`

4. `practice/chapter6/results/6-5-rerun-rollup.log`

☐ 이어서 네 가지를 각각 세 문장 이내로 쓴다

1. 6-3 로그의 [4] 에서 **날짜 하나**를 골라 `physical · dup · valid · mapped · missing · unmapped` 를 옮겨 적고, 총계 보존식이 성립함을 산술로 보인다.

2. `missing` 과 `unmapped` 를 합쳐 "실패 2건"으로 보고하면 안 되는 이유를, 두 항의 조치 대상과 연결해 쓴다.

3. 6-4 로그의 [3] 에서 실행된 태스크와 실행되지 않은 태스크를 나눠 적고, 남은 산출물이 0개인 것이 왜 중요한지 쓴다.

4. 6-5 로그의 [5] 에서 두 매핑 건수를 옮겨 적고, 두 보고서 모두 총계 보존이 성립하는데도 문제인 이유를 쓴다.

☐ 값을 지어내지 않는다

☐ 실행이 안 되면 어디서 막혔는지 그대로 쓴다

☐ 수업일 포함 7일 이내에 제출한다